

CURSO: PIAD-525_DISEÑO Y DESARROLLO DE APLICACIONES MÓVILES

Tarea – HT-04

Usa y crea aplicaciones con API REST.

Operaciones:

1. Consume servicios de API RESTful en Android Studio.
2. Crea aplicación de caso práctico.
3. Identifica requerimientos para la publicación de una App.

Objetivo de la Tarea

Al concluir la tarea el participante estará en condiciones de diseñar y desarrollar una aplicación en Android Studio que permita al usuario final consumir el servicio RESTful. Además, se definirán los requisitos necesarios para su futura publicación en Google Play.

Caso Práctico

Una empresa de servicios financieros, FinCo, necesita una aplicación móvil para Android que permita a sus empleados iniciar sesión de manera segura y acceder a su perfil y datos esenciales desde cualquier ubicación. La aplicación debe comunicarse con una API segura que gestione la autenticación y entregue datos personalizados para cada usuario.

Por lo que se requiere que se desarrolle: Consume servicios de API RESTful en Android Studio, crea aplicación de caso práctico e identifica requerimientos para la publicación de una App

Materiales/ Instrumentos/ Equipos/Herramientas/ Reactivos/ Insumos/ Colorantes.

Las siguientes listas son de referencia.

El instructor puede variar los requerimientos, con fin de desarrollar la tarea.

Materiales:

Nombre	Cantidad
Documentación de Retrofit y API REST	1
Cuenta de Desarrollador de Google Play	1

Instrumentos y Equipos:

Nombre	Cantidad
Computadora/Laptop con 16gb de memoria RAM	1
Dispositivo Android físico (o emulador)	1
Conexión a Internet	1

Herramientas:

Nombre	Cantidad
Android Studio	1
Java Development Kit (JDK)	1

Desarrollo de la Práctica

OPERACIÓN 01: Consume servicios de API RESTful en Android Studio.

Paso 1: Configuración del Proyecto en Android Studio

- Crear un nuevo proyecto en Android Studio.

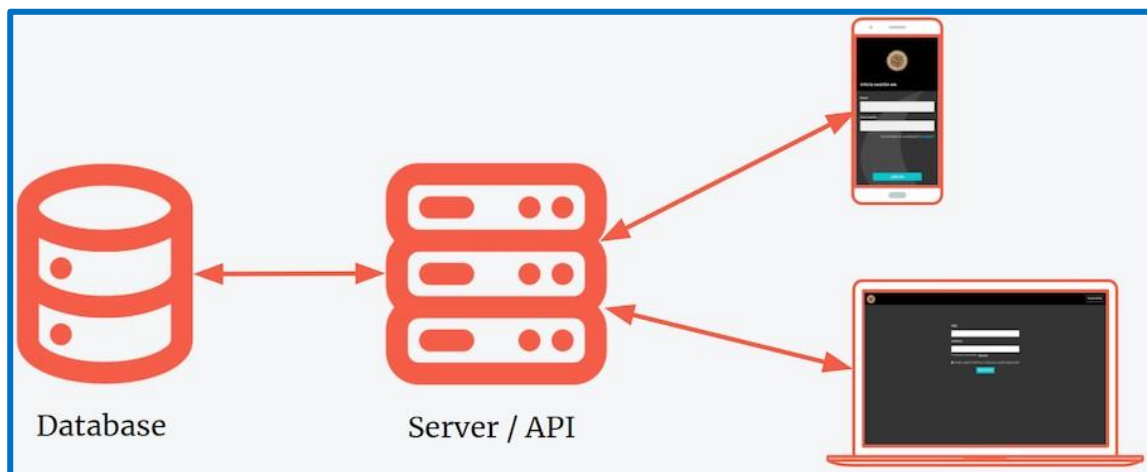
○ **Nombre del proyecto:** FinCoLoginApp. ○

Actividad inicial: Empty Activity.

- Agregar dependencias en build.gradle:

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0' implementation  
'com.squareup.retrofit2:converter-gson:2.9.0' implementation  
'com.squareup.okhttp3:logging-interceptor:4.9.0'
```

```
// _____  
// RETROFIT 2 - Consumir API RESTful de FinCo  
// _____  
implementation("com.squareup.retrofit2:retrofit:2.9.0")  
implementation("com.squareup.retrofit2:converter-gson:2.9.0")  
implementation("com.squareup.okhttp3:logging-interceptor:4.9.0")
```



Diseño de inicio de sesión que se implementará en el proyecto.

Paso 2: Configuración de Retrofit para Consumo de API

- Crear una clase ApiClient para gestionar la conexión con la API:

```
import retrofit2.Retrofit; import  
retrofit2.converter.gson.GsonConverterFactory;  
  
public class ApiClient {    private  
    static Retrofit retrofit = null;
```

```

    public static Retrofit getClient(String baseUrl) {
    if (retrofit == null) {          retrofit = new
    Retrofit.Builder()
        .baseUrl(baseUrl)
        .addConverterFactory(GsonConverterFactory.create())
        .build();
    }
    return retrofit;
    }
}

```

- Crear una clase ApiService que defina los endpoints:

```

import retrofit2.Call; import
retrofit2.http.Body;
import retrofit2.http.POST;

```

```

public interface ApiService {
    @POST("login")
    Call<LoginResponse> loginUser(@Body LoginRequest request);
}

```

```

package com.example.apirest;

import okhttp3.OkHttpClient;
import okhttp3.logging.HttpLoggingInterceptor;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;
import java.util.concurrent.TimeUnit;

1 usage
public class ApiClient {
    4 usages
    private static Retrofit retrofit = null;
    no usages
    private ApiClient() {}

    1 usage
    public static Retrofit getClient(String baseUrl) {

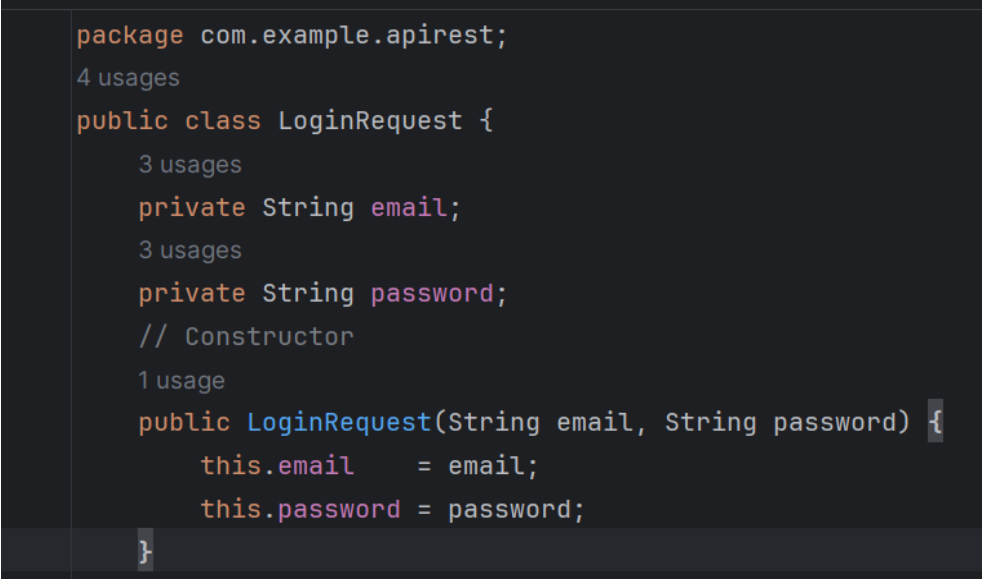
        if (retrofit == null) {

```

Paso 3: Lógica para Autenticación

- Crear la clase LoginRequest para representar los datos de inicio de sesión:

```
public class LoginRequest {  
    private String email;  
    private String password;  
  
    public LoginRequest(String email, String password) {  
        this.email = email;    this.password = password;  
    }  
}
```



```
package com.example.apirest;  
4 usages  
public class LoginRequest {  
    3 usages  
    private String email;  
    3 usages  
    private String password;  
    // Constructor  
    1 usage  
    public LoginRequest(String email, String password) {  
        this.email    = email;  
        this.password = password;  
    }  
}
```

- Crear la clase LoginResponse para manejar la respuesta del servidor:

```
public class LoginResponse {  
    private String token;    private  
    String message;  
    private boolean success;  
  
    // Getters y setters  
}
```

```

package com.example.apirest;

import com.google.gson.annotations.SerializedName;
7 usages
public class LoginResponse {
    4 usages
    private String token;
    2 usages
    @SerializedName(value = "message", alternate = {"error"})
    private String message;
    2 usages
    private boolean success;

    // Getters
    1 usage
    public String getToken() { return token; }
    no usages
    public String getMessage() { return message; }

```

- Crear el método para el inicio de sesión en la actividad:

```

ApiService                                apiService                                =
ApiClient.getClient("https://api.fingo.com/").create(ApiService.class);
LoginRequest request = new LoginRequest("email@example.com", "password123");

Call<LoginResponse> call = apiService.loginUser(request); call.enqueue(new
Callback<LoginResponse>() {
    @Override
    public void onResponse(Call<LoginResponse> call, Response<LoginResponse>
response) {    if (response.isSuccessful()) {
        // Procesar respuesta de éxito
    } else {
        // Procesar error
    }
}
    @Override    public void onFailure(Call<LoginResponse> call,
Throwable t) {    // Manejar error de conexión
    }
});

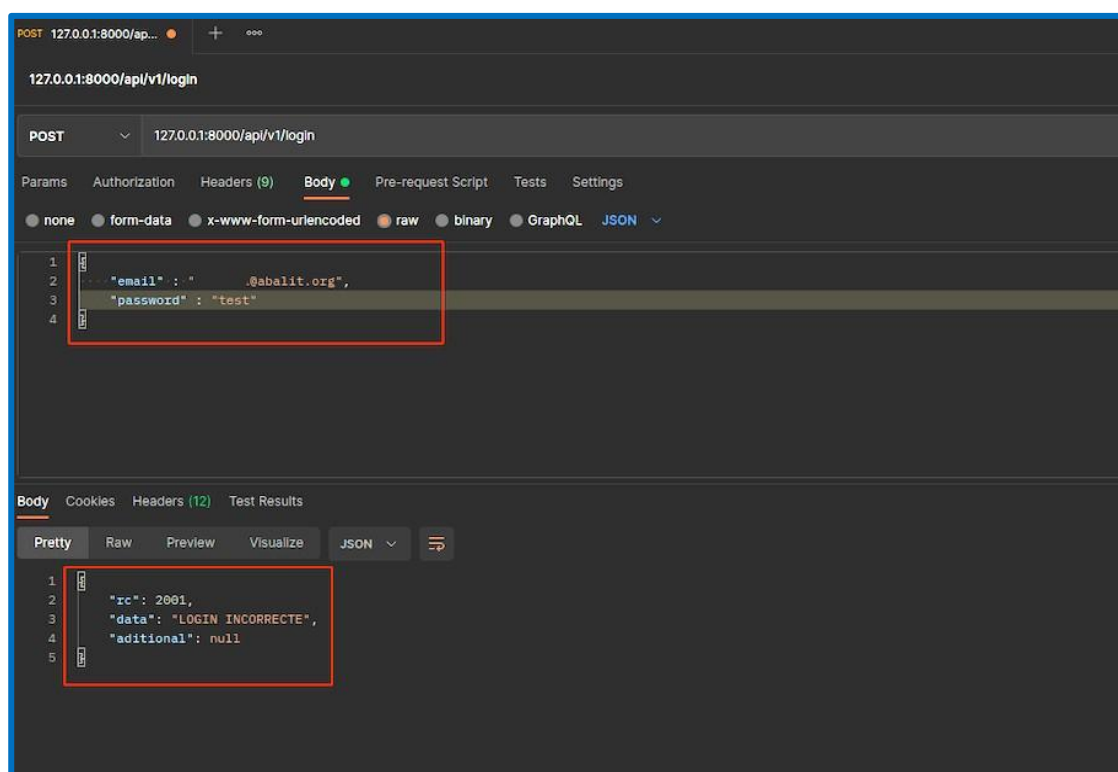
```

```

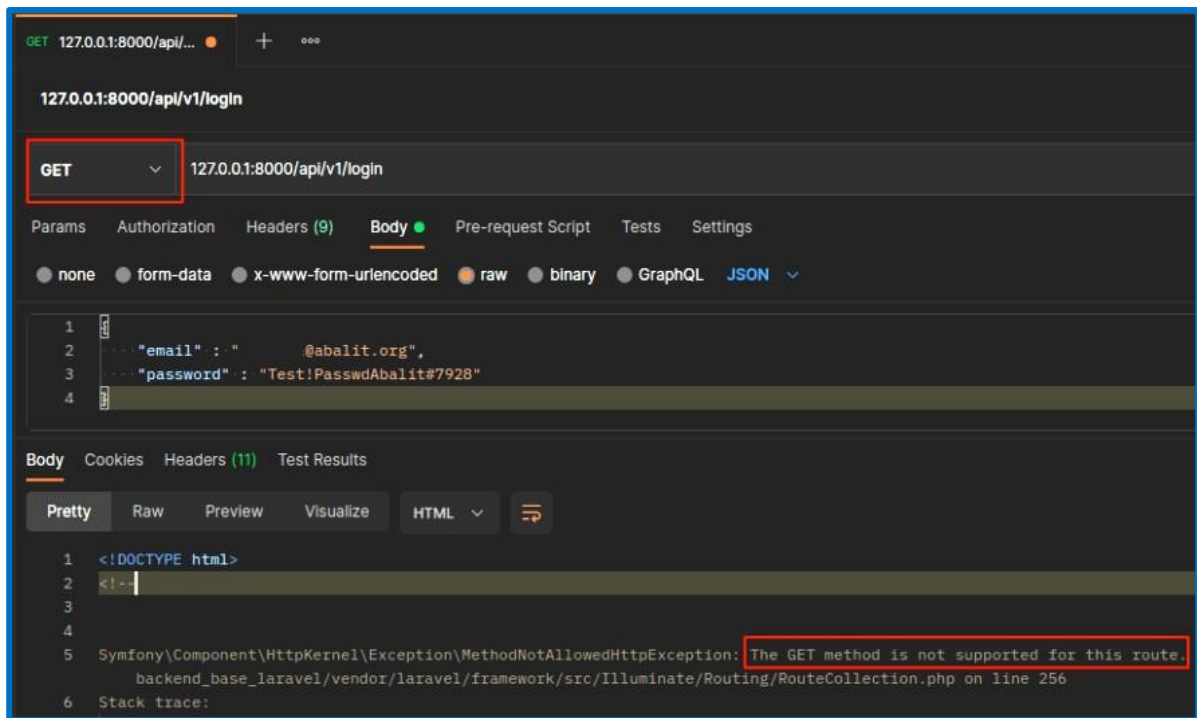
import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.GET;
import retrofit2.http.Header;
import retrofit2.http.POST;

2 usages
public interface ApiService {
    1 usage
    @POST("login")
    Call<LoginResponse> loginUser(@Body LoginRequest request);
    no usages
    @POST("register")
    Call<LoginResponse> registerUser(@Body LoginRequest request);
    no usages
    @GET("users/2")
    Call<UserProfileResponse> getUserProfile(
        @Header("Authorization") String token
    );
}

```



[Captura de pantalla del formulario de inicio de sesión en la aplicación](#)



Error por método HTTP incorrecto

OPERACIÓN 02: Crea aplicación de caso práctico.

Paso 1: Diseño de la Interfaz

Crear diseño de la interfaz de usuario para inicio de sesión en `activity_login.xml`:

```

<EditText
    android:id="@+id/emailEditText"
    android:hint="Email"
    android:inputType="textEmailAddress" />

```

```

<EditText
    android:id="@+id/passwordEditText"
    android:hint="Password"
    android:inputType="textPassword" />

```

```

<Button
    android:id="@+id/loginButton"
    android:text="Login" />

```




Paso 2: Lógica de validación de datos en LoginActivity.java:

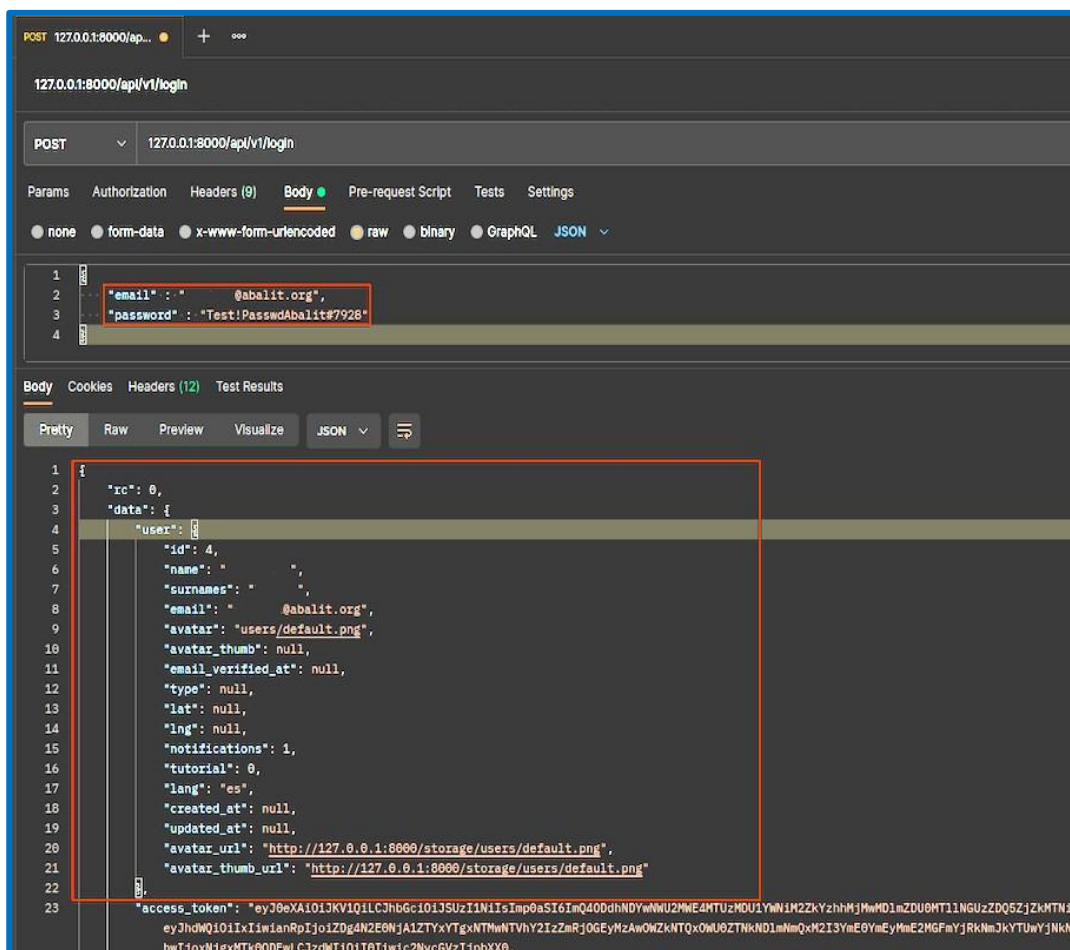
Validar que los campos no estén vacíos antes de enviar la solicitud de autenticación.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    emailEditText = findViewById(R.id.emailEditText);
    passwordEditText = findViewById(R.id.passwordEditText);
    loginButton = findViewById(R.id.loginButton);
    progressBar = findViewById(R.id.progressBar);

    loginButton.setOnClickListener( View v -> validarEntrada());

    findViewById(R.id.switchModoTextView).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: MainActivity.this, RegisterActivi
    });
}
```



Elementos de entrada de texto y el botón de inicio de sesión.

11:40

94

FinCo
PANEL DE CONTROL INTERNO

Nombres
YEYSON

Apellidos
CHAMORRO

Edad Estimada
18 años

Correo Electrónico
jeysonchamorroarias@gmail.com

ID Sesión: TOKEN_LOCAL_GMAIL_1772901616178

CERRAR SESIÓN

FinCo
REGISTRO DE EMPLEADO

Nombres

Apellidos

Ej: Juan Antonio

Ej: Pérez García

Fecha de Nacimiento

Seleccione fecha

Correo Electrónico

ejemplo@empresa.com

Contraseña

Confirmar Contraseña

REGISTRAR EMPLEADO

[¿Ya tienes cuenta? Inicia sesión](#)

OPERACIÓN 03: Identifica requerimientos para la publicación de una App.

Para publicar la aplicación en Google Play, se deben seguir estos pasos:

- **Crear una cuenta de desarrollador de Google** (pago único de \$25 USD).
- **Configurar la seguridad:** asegurarse de manejar adecuadamente los datos de usuarios y tokens de API.
- **Preparar el APK/AAB:** generar el archivo AAB desde Android Studio.
- **Configurar la descripción:** proporcionar información sobre las características y requisitos de la aplicación.
- **Crear gráficos de la app:** imágenes de la aplicación para la galería en Play Store.

Actividades para el Estudiante

1. ¿Por qué es importante validar las credenciales de usuario en el lado del servidor?

Aunque en Android guardes datos, la seguridad real está en el servidor. Si alguien piratea el celular o intercepta la App, no podrá entrar a tu base de datos si el servidor pide un Token válido cada vez. Es la única forma de proteger los cálculos de los datos.

2. ¿Qué conocimientos de programación y diseño fueron más útiles para desarrollar la aplicación?

Java (Objetos): Para crear los modelos de datos que representan tus proyectos.

XML y Material Design: Para que las tablas de se vean bien en cualquier pantalla.

Ciclo de Vida (Activity): Evitar que la App se cierre o pierda los datos cuando giras el celular o sales un momento.

3. ¿Qué problemas técnicos encontraste en la conexión con la API y cómo los resolviste?

Tráfico HTTP bloqueado: Android no deja conectar a servidores locales por seguridad. Lo arreglaste configurando el `networkSecurityConfig` en el Manifest.

Errores de JSON: A veces la API mandaba datos que no encajaban. Lo solucionaste con **Gson** para mapear todo correctamente.

App congelada: Las peticiones a internet bloqueaban la pantalla. Lo arreglaste usando **Coroutines** para que la red trabaje en segundo plano.

4. ¿Qué habilidades nuevas o conocimientos obtuviste al trabajar con Retrofit y la API REST?

Aprendiste a estructurar peticiones de forma profesional (GET, POST), a usar **Interceptores** para enviar el token de seguridad automáticamente y a manejar errores (como el 404 o 500) para que la App no "explote" si falla el internet.

5. ¿Qué nuevas funcionalidades considerarías en una futura actualización de la aplicación para mejorar el seguimiento de envíos?

Notificaciones Push: Para avisar al toque cuando hay un cambio en el proyecto.

Google Maps: Para ver la ubicación real de las cajas de registro.

Modo Offline: Para poder trabajar sin señal en el campo y que todo se sincronice al detectar Wi-Fi.